Name: Asmit Pandey
Roll No. 2401072
Class: T21

# EXPERIMENT NO.: 05

**AIM:**

To set up a Continuous Integration (CI) Pipeline using Jenkins and GitHub Actions for automatically building and testing an application.

**THEORY:**

Continuous Integration (CI) is a software development practice in which developers frequently integrate their code changes into a shared repository. Whenever new code is pushed to the repository, an automated CI pipeline can perform tasks such as building the application, running tests, and checking whether the code works correctly.

· Continuous Integration helps developers detect errors early.

· Developers push their source code to a version control system such as GitHub.

· A CI tool automatically detects changes made to the source code repository.

· The application is automatically built and tested.

· If all tests are successful, the pipeline is marked as successful.

· If a build or test fails, the pipeline reports a failure.

· Jenkins and GitHub Actions are commonly used tools for implementing Continuous Integration.

**Jenkins** is an automation server that allows developers to create pipelines for building, testing, and deploying applications.

**GitHub Actions** is a CI/CD automation platform integrated directly with GitHub repositories. Workflows can automatically run when events such as a code push or pull request occur.

A typical Continuous Integration process is represented as:

Developer
↓
Push Code to GitHub

Name: Asmit Pandey
Roll No. 2401072
Class: T21

↓
CI Pipeline Triggered
↓
Install Dependencies
↓
Build Application
↓
Run Tests
↓
Success / Failure Report

## Network / Software Setup

**Components Required**

· Computer/Laptop

· Jenkins

· Git

· GitHub Account

· Node.js Application (Test Application)

· Internet Connection

· GitHub Repository

## Sample Application Structure

test-application/
│
├── package.json
├── index.js
├── test/

Name: Asmit Pandey
Roll No. 2401072
Class: T21

```
│   └── app.test.js
│
├── Jenkinsfile
│
└── .github/
    └── workflows/
        └── ci.yml
```

## Sample Application index.js

```
function add(a, b) {
return a + b;
}

module.exports = { add };
```

## package.json

```
{
  "name": "jenkins-ci-demo",
  "version": "1.0.0",
  "scripts": {
    "test": "node test/app.test.js"
  }
}
```

## test/app.test.js

```
const { add } = require("../index");
```

Name: Asmit Pandey
Roll No. 2401072
Class: T21

```
if (add(2, 3) !== 5) {
console.error("Test Failed!");
process.exit(1);
}

console.log("Test Passed Successfully!");
```

## Jenkins Pipeline Setup

## Jenkinsfile

Create a file named Jenkinsfile in the root directory of the project.

```
pipeline {
agent any

   stages {

      stage('Checkout') {
steps {
checkout scm
         }
      }

      stage('Install Dependencies') {
steps {              sh 'npm install'
         }
      }

      stage('Run Tests') {
steps {              sh
'npm test'
         }
      }
```

Name: Asmit Pandey
Roll No. 2401072
Class: T21

```
        stage('Build Status') {            steps {                echo 'Continuous
Integration Pipeline Completed Successfully!'
            }
        }
    }
}
```

The Jenkins pipeline performs the following operations:

Checkout Source Code
↓
Install Dependencies
↓
Run Tests
↓
Generate Build Status

**GitHub Actions Setup**

Create the following directory structure:

.github/workflows/

Inside the workflows folder, create a file named:

ci.yml

## GitHub Actions Workflow

```
name: Continuous Integration

on:   push:
branches:
- main
```

Name: Asmit Pandey
Roll No. 2401072
Class: T21

```
 pull_request:
branches:
- main

jobs:   build-and-
test:

   runs-on: ubuntu-latest

   steps:

- name: Checkout Repository
  uses: actions/checkout@v4

- name: Setup Node.js        uses:
  actions/setup-node@v4

- name: Install Dependencies
  run: npm install

- name: Run Tests        run: npm test
```

The GitHub Actions workflow is automatically triggered when:

· Code is pushed to the main branch.

· A pull request is created for the main branch.

**Procedure**

1. Open the computer and ensure that Git, Jenkins, and Node.js are installed.
2. Create a simple test application.
3. Create the required project files such as index.js, package.json, and the test file.
4. Test the application locally using:

```
npm test
```

Name: Asmit Pandey
Roll No. 2401072
Class: T21

**5.** Create a GitHub repository for the application.

**6.** Initialize the local project as a Git repository.

git init

**7.** Add the project files to Git.

git add .

**8.** Commit the source code.

git commit -m "Initial CI application"

**9.** Connect the local repository to GitHub.

**10.** Push the project to the GitHub repository.

## Jenkins Configuration Procedure

**11.** Start the Jenkins server.

**12.** Open Jenkins in a web browser.

http://localhost:8080

**13.** Click on **New Item**.

**14.** Enter a name for the project.

Example:

CI-Pipeline-Demo

Name: Asmit Pandey
Roll No. 2401072
Class: T21

**15.** Select **Pipeline** as the project type.

**16.** Click **OK**.

**17.** In the Pipeline configuration, select:

Pipeline script from SCM

**18.** Select:

Git

as the Source Code Management system.

**19.** Enter the GitHub repository URL.

**20.** Select the required branch, such as:

*/main

**21.** Enter the script path:

Jenkinsfile

**22.** Save the pipeline configuration.

**23.** Click **Build Now** to execute the Jenkins pipeline.

**24.** Observe the different stages of the Jenkins pipeline.

## GitHub Actions Procedure

**25.** Create a .github/workflows directory in the project repository.

**26.** Create the GitHub Actions workflow file named:

ci.yml

Name: Asmit Pandey
Roll No. 2401072
Class: T21

**27.** Add the Continuous Integration workflow configuration.

**28.** Commit and push the workflow file to GitHub.

```
git add .
git commit -m "Add CI workflow" git
push origin main
```

**29.** Open the GitHub repository.

**30.** Navigate to the **Actions** tab.

**31.** Observe the automatically triggered workflow.

**32.** Verify the following CI stages:

Checkout Repository
↓
Setup Node.js
↓
Install Dependencies
↓
Run Tests

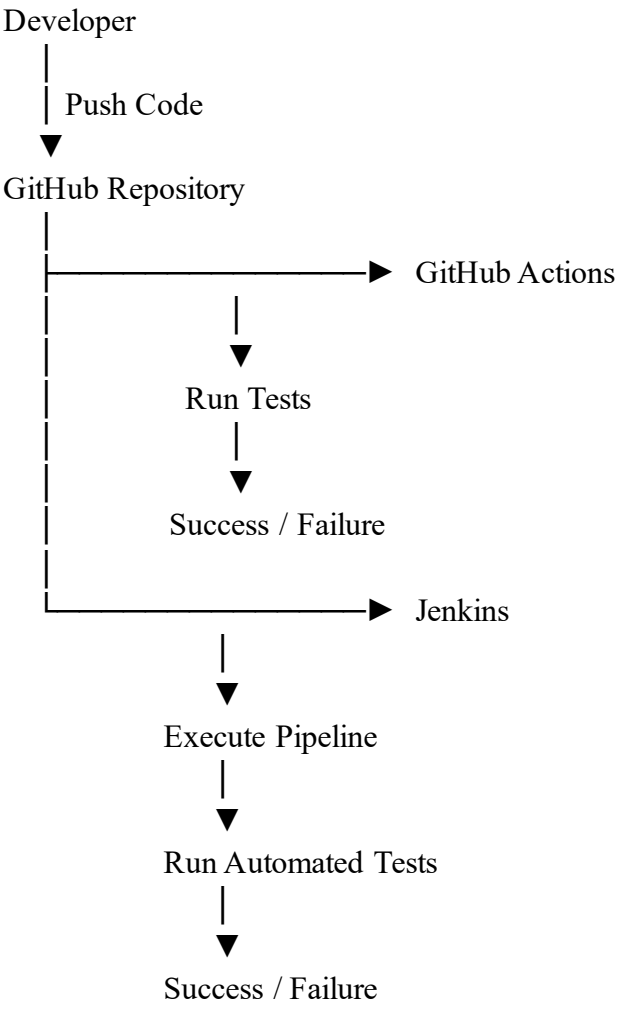
## CI Pipeline Observation

| Step | CI Stage | Operation | Observation |
|---|---|---|---|
| 1 | Source Code | Code pushed to GitHub | Repository receives updated source code |
| 2 | Pipeline Trigger | Jenkins/GitHub Actions starts | CI process begins automatically or manually |
| 3 | Checkout | Source code is downloaded | Latest version of application is obtained |
| 4 | Dependencies | Required packages are installed | Application environment is prepared |
| 5 | Testing | Automated tests are executed | Application functionality is verified |

Name: Asmit Pandey
Roll No. 2401072
Class: T21

| | | | |
|---|---|---|---|
| 6 | Result | Pipeline status generated | Build is marked as Success or Failure |

### Continuous Integration Workflow




# Expected Result

· The application source code is successfully stored in the GitHub repository.

· Jenkins successfully checks out the source code.

Name: Asmit Pandey
Roll No. 2401072
Class: T21

· The CI pipeline installs the required dependencies.

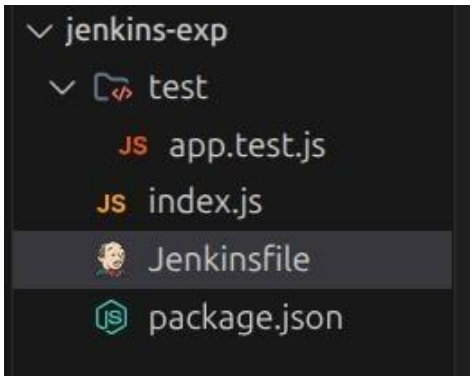
· Automated tests are executed successfully.

· Jenkins displays the status of the pipeline.

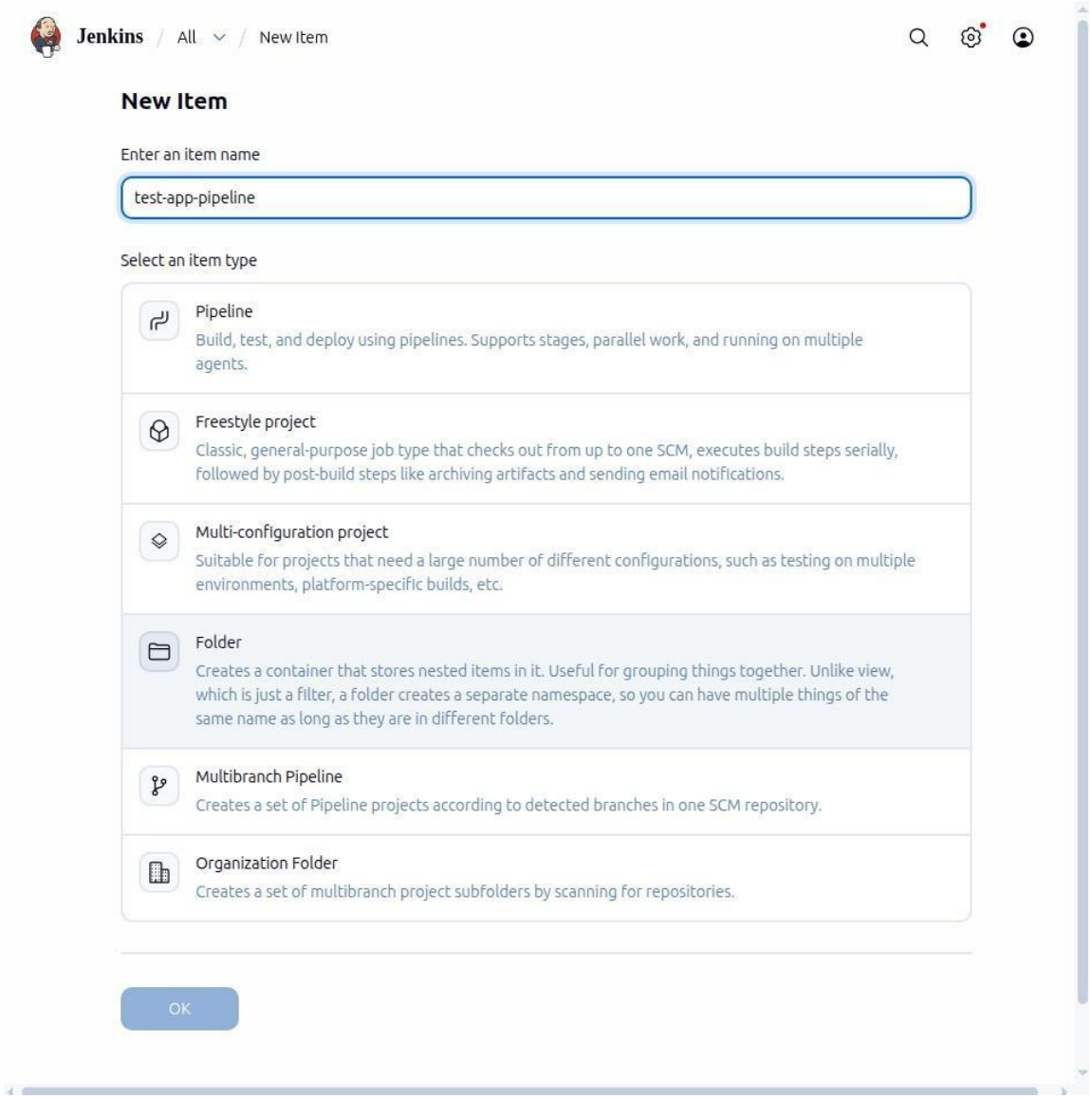
· GitHub Actions automatically runs the CI workflow after a push or pull request.

· The workflow status is displayed as either successful or failed.

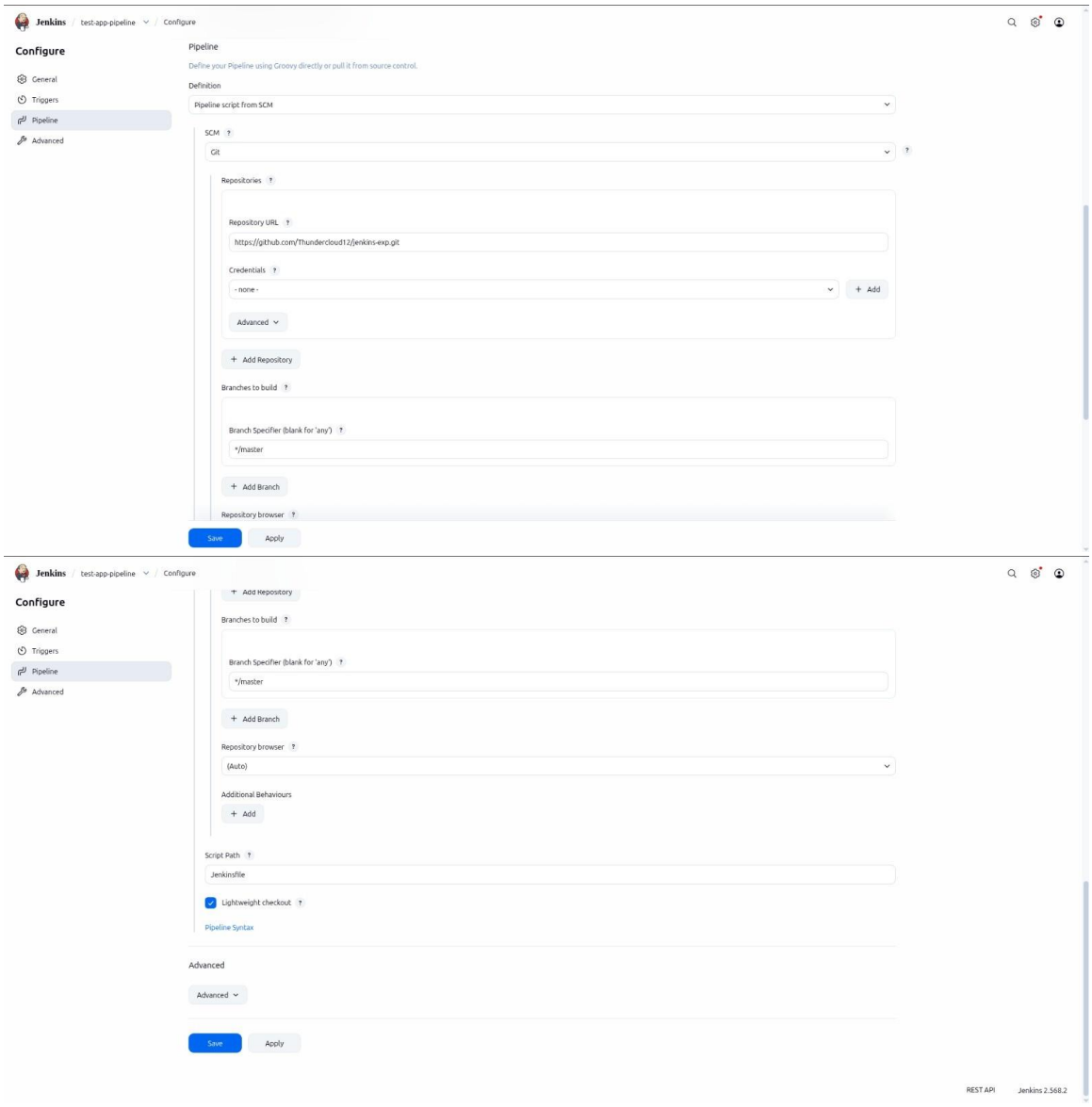
## Screenshot:



Name: Asmit Pandey
Roll No. 2401072
Class: T21

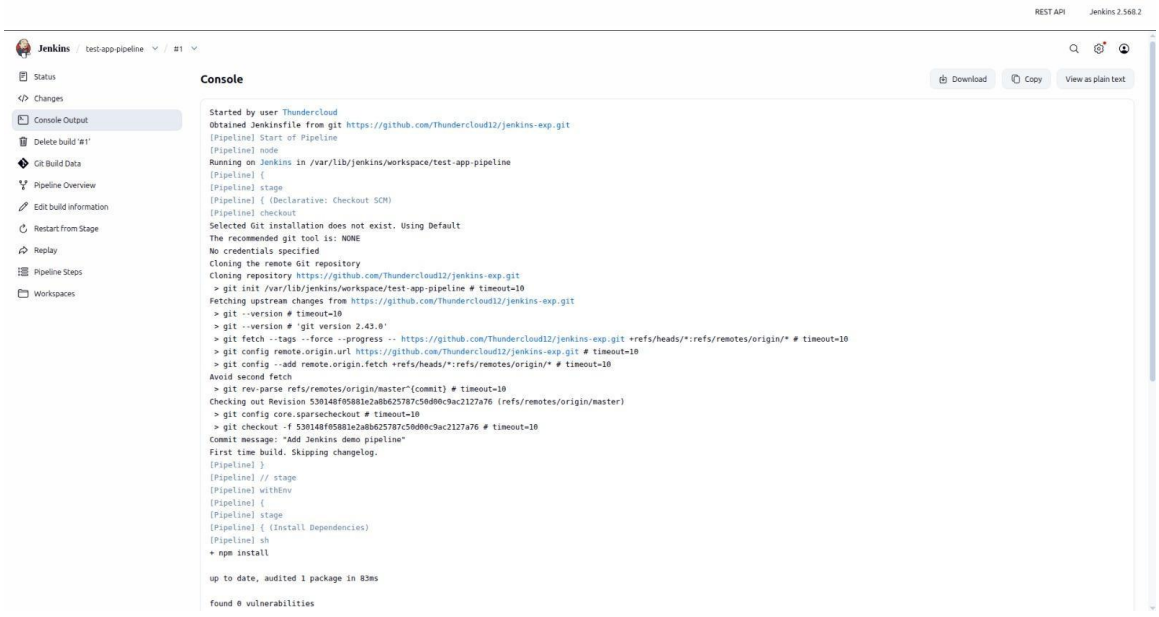


Name: Asmit Pandey
Roll No. 2401072
Class: T21

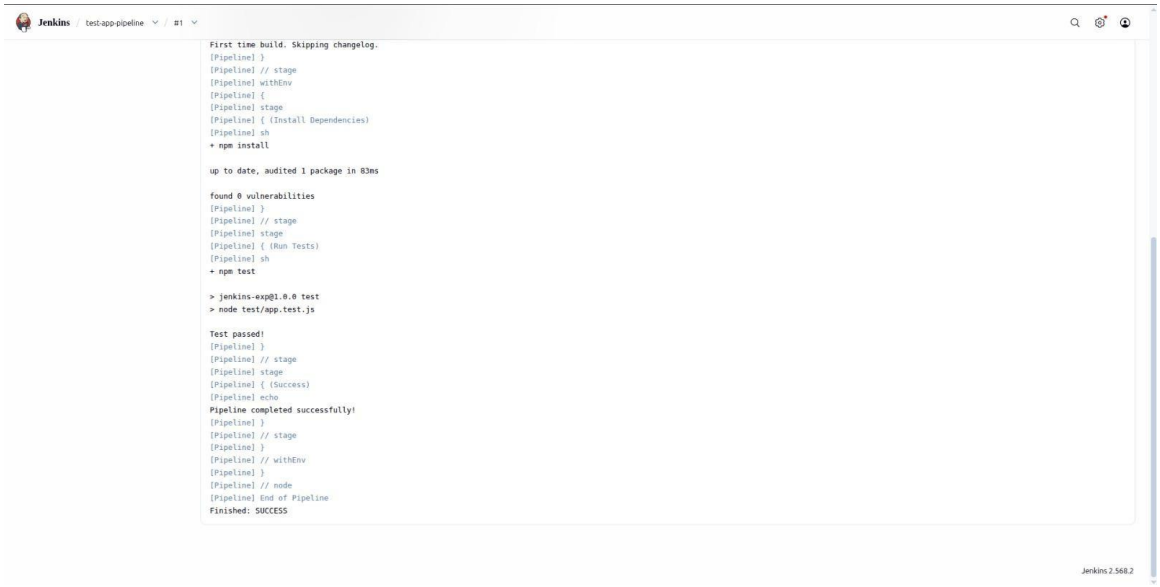


Name: Asmit Pandey
Roll No. 2401072
Class: T21



Name: Asmit Pandey
Roll No. 2401072
Class: T21




## Result:

The Continuous Integration Pipeline was successfully set up using Jenkins and GitHub Actions. The application source code was automatically checked out, dependencies were installed, and automated tests were executed. The pipeline successfully reported the build status.

## CONCLUSION:

Thus, a Continuous Integration Pipeline was successfully implemented using Jenkins and GitHub Actions. The experiment demonstrated how source code changes can be automatically integrated, built, and tested using a CI pipeline. This process helps identify errors early and improves the reliability and quality of software development.